

Real-Time Human and Generative AI Interaction: Network Challenges and Opportunities

Ruizhi Cheng , Guowu Xie , and Bo Han 

ABSTRACT

The rapid rise of real-time, audio/video-based interactions between humans and generative AI (GenAI) introduces fundamental challenges for today's network infrastructure. Our measurement study, combined with an in-depth analysis of existing deployments, reveals two key shortcomings in the current system design for human and GenAI interaction. First, they fall short of providing fast and robust network adaptation, resulting in a degraded user experience under dynamic and lossy network conditions. Second, existing platforms adopt inconsistent and ad hoc transport-layer strategies to reuse legacy architectures that are ill-suited for the unique traffic patterns of human and GenAI interactions. To address these challenges, we first propose leveraging the network edge as an intelligent relay between users and GenAI backends to perform key network adaptation tasks such as congestion control. This architecture shortens the length of the path that network adaptation mechanisms must handle, improving their effectiveness. We further analyze the tradeoffs between various transmission paradigms across different network segments and identify the proper strategy for each. Preliminary results from our proof-of-concept implementation demonstrate the feasibility of this architecture for empowering real-time human and GenAI interaction.

INTRODUCTION

The emergence of voice/video-based interactions between human users and multimodal large language models (LLMs) marks a transformative shift in how users engage with generative artificial intelligence (GenAI). Moving beyond the traditional text interface, GenAI applications such as OpenAI ChatGPT¹ and Meta AI² are pioneering a new paradigm: real-time human and GenAI interaction (human-GenAI interaction, for short) [2]. With these applications, users can converse with GenAI using natural speech, optionally augmented with video input, via a range of devices such as smartphones, tablets, and smart glasses (e.g., Meta's Ray-Ban Glasses³). These applications enable rich, multimodal experiences, enabling various use cases such as contextual assistance (e.g., helping users navigate with smart glasses.⁴

As this paradigm gains traction, it demands a rethinking of the underlying network infrastructure. Traditionally, media exchange/delivery over the Internet has been optimized via two dominant paradigms: (1) real-time communication (RTC) [10], which primarily supports bidirectional, latency-sensitive streaming such as audio and video calling [9]; and (2) content delivery networks (CDNs) [7], which are designed for scalable distribution of content via on-demand video streaming [15] or live broadcasting [7], emphasizing caching efficiency and bandwidth optimization.

Human-GenAI interaction, however, both blends and diverges from these paradigms. Specifically, the *uplink* consists of continuous, real-time user input, closely resembling RTC. However, different from RTC, where endpoints are typically behind NAT (network address translation) [9], human-GenAI interactions involve clients connecting to backend servers (e.g., data centers [12]) that have static, globally routable IP addresses. Thus, client connections can be initiated directly, eliminating session-establishment overhead in RTC protocol stacks such as ICE/STUN/TURN.⁵ On the other hand, the *downlink* carries GenAI responses, generated in tokenized bursts rather than continuous streams [8], and can be synthesized into audio before playback begins. Thus, the downlink may not require RTC-like transmission strategies that deliver content frame by frame. However, CDN-style solutions are also insufficient, as GenAI responses are dynamically generated, highly personalized, and non-cacheable due to context sensitivity and privacy concerns. Therefore, the unique feature of human-GenAI interaction disrupts the foundational assumptions of both RTC and CDN.

In this paper, we argue that human-GenAI interaction requires reimagining existing infrastructures, for example, carefully integrating elements of both RTC and CDN while accommodating the unique characteristics of GenAI traffic, to ultimately enable low-latency, flexible, and efficient GenAI-mediated communication systems. Specifically, to ensure high quality of experience (QoE), the underlying network must simultaneously meet three critical requirements:

- To preserve a natural and fluid conversational experience for users, the end-to-end

¹ <https://openai.com/index/chatgpt/> (accessed on 30-September-2025)

² <https://www.meta.ai/> (accessed on 30-September-2025)

³ <https://www.meta.com/ai-glasses/ray-ban-meta/> (accessed on 30-September-2025)

⁴ <https://tinyurl.com/5d-wrb9sn> (accessed on 30-September-2025)

⁵ <https://datatracker.ietf.org/doc/html/rfc8835> (accessed on 30-September-2025)

delay, from user input to GenAI response, must remain within sub-second bounds.⁶

- Given the prevalence of mobile and wireless access, the network support must gracefully handle diverse network conditions such as fluctuating bandwidth and packet losses.
- The underlying protocols must accommodate directionally asymmetric traffic without imposing a heavy overhead.

Our measurement study of popular human-GenAI interaction applications (the section “[Motivation](#)”), alongside insights from large-scale LLM deployments [12], reveals that current network support falls short of meeting the aforementioned requirements. As illustrated in Figure 1(a), human-GenAI interaction applications face three critical challenges: **First**, LLM inference is typically performed in data centers [12]. Due to resource constraints and policy restrictions,⁷ only a limited number of such data centers exist globally, leading to high round-trip times (RTTs) for users who are geographically distant from these data centers. **Second**, GenAI-generated content may demand up to ~4 Mbps bandwidth (the section “[Motivation](#)”), making network adaptation such as congestion control essential [9]. However, a common practice is to perform these adaptations at remote data centers, which is suboptimal, as network adaptation is highly sensitive to RTT (the section “[Preliminary Results](#)”). **Third**, existing GenAI platforms exhibit significant diversity in their choices of transmission paradigms and protocol stacks (the section “[Motivation](#)”). This inconsistency underscores the lack of consensus and the open question regarding the most suitable networking design for human-GenAI interaction.

To address these challenges, we propose a holistic network architecture for optimizing human-GenAI interaction, as shown in Figure 1(b). In the following, we describe the key efforts that inform this design, as our main contributions.

- We propose leveraging the network edge (e.g., the CDN node) as an intelligent relay between users and LLM backends, hosting the network adaptation layer (NADL) to manage transmission dynamics such as network adaptation (the section “[Leveraging Edge for Network Adaptation](#)”). By positioning the NADL to the edge, the bottleneck link over which network adaptation operates is reduced to the short client ↔ edge path, rather than the much longer client ↔ data center path in the existing deployment, drastically improving the responsiveness and efficiency of network adaptation mechanisms (the section “[Preliminary Results](#)”).
- We develop a direction-dependent delivery framework that judiciously selects the most appropriate transmission paradigm for each segment of the end-to-end path and propose a unified transport protocol design to support asymmetric and adaptive communication (the section “[Direction-Dependent Delivery Method](#)”). For the uplink, the RTC-style frame-based approach is preferable, as demonstrated in our measurements (the section “[Motivation](#)”). For the downlink, chunk-based solution is well-suited for the high-throughput, stable data center-to-edge segment. In contrast, for the edge-to-client segment, which traverses the public Internet with variable conditions, the optimal delivery strategy depends on both network characteristics and user interaction patterns. To enable this adaptivity, we introduce a QoE-driven model to select the delivery method within the NADL. We further explore transport-layer protocol designs that facilitate seamless switching between different choices.
- Finally, we build a proof-of-concept to evaluate the benefits of our design (the section “[Preliminary Results](#)”). We assess three widely used congestion control algorithms under real-world cellular network loss trace, across links with varying RTT. Our results demonstrate that compared to high RTT, low RTT consistently yields better performance, with higher bandwidth utilization, lower throughput variability, and faster recovery to the target sending rate after loss.

BACKGROUND

Real-time Communication (RTC) [10] is a transmission paradigm to support interactive live applications such as audio/video calling [9] and immersive telepresence [3], [4]. RTC systems are optimized to minimize end-to-end latency, which is critical for enabling seamless and timely interaction among participants. For instance, it typically utilizes UDP at the transport layer, which is widely recognized for offering lower latency compared to TCP. Moreover, to minimize latency, RTC systems adopt a frame-based approach, transmitting content frame by frame as it becomes available. This design relies on stable bandwidth availability to ensure consistent quality across frames.

Content Delivery Networks (CDNs) are network infrastructures for the scalable distribution of media content, supporting one-to-many live broadcasting [7] and on-demand video

⁶ <https://www.itu.int/rec/t-rec-g.114-200305-i> (accessed on 30-September-2025)

⁷ <https://tinyurl.com/azdc86nd> (accessed on 30-September-2025)

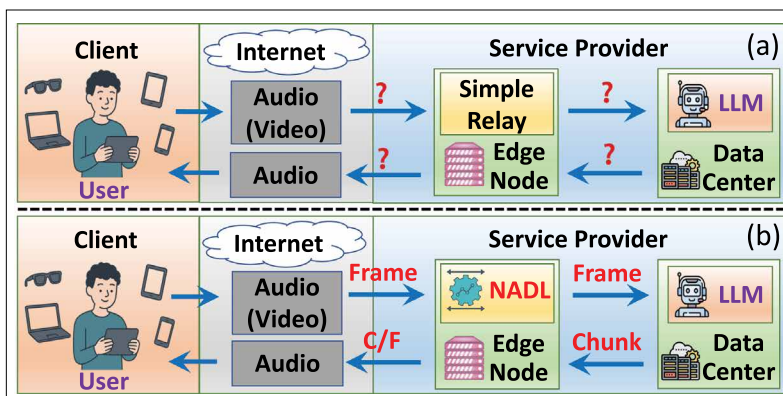


FIGURE 1. The workflow of a) existing and b) proposed real-time human and GenAI interaction. Client-side video input is optional. a) In current deployments, the communication between the user and the large language model (LLM) in the data center is routed through an edge node acting as a simple relay. Moreover, the appropriate transmission paradigm for uplink and downlink remains unclear (the section “[Motivation](#)”). b) We propose placing the network adaptation layer (NADL) at the edge as an intelligent relay. NADL performs network adaptation and selects the optimal transmission strategy per link: frame-based for uplink, chunk-based from the data center to the edge, and adaptive (chunk- or frame-based) from the edge to the user.

streaming [15]. A CDN consists of geographically distributed nodes deployed at the network edge to serve users in close proximity. A core function of CDN nodes is to cache popular content, enabling efficient reuse across multiple users and thus reducing network load. Accordingly, CDN-based systems prioritize caching efficiency and bandwidth utilization. To achieve this, CDN-based streaming typically employs the chunk-based method that downloads multiple data chunks (i.e., segments) in succession via reliable transport protocols such as TCP.

Real-time Human and GenAI Interaction [2] involves a client-side application and a remote LLM backend deployed in data centers [12], as shown in Figure 1(a). On the client side, users can employ various types of devices. The captured audio (and video) is transmitted to the LLM backend, which processes the inputs and returns generated audio responses.

This interaction paradigm introduces a new content delivery model that does not align cleanly with existing categories. At first glance, it may resemble a real-time audio/video conversation between a human and an LLM-based “virtual human”. This may hold for the uplink, as the user’s speech (and video) is captured and streamed in real time. Thus, the uplink transmission resembles RTC.

The transmission paradigm for downlink merits further scrutiny. Different from human-generated audio that produced continuously in real time, GenAI responses are generated in a tokenized format, where each token represents a portion of the response. These tokens are often emitted in bursts rather than at uniform intervals [8], and can be rapidly synthesized into audio before playback begins. Thus, although the audio is ultimately played back at a steady rate, much or all of it may be available in advance of transmission. As a result, frame-by-frame RTC-style streaming may not be optimal. Instead, chunk-based delivery, similar to CDN workflows, can be a viable alternative. However, CDN techniques offer limited utility here, as LLM responses are highly personalized and *non-cacheable* due to privacy and context sensitivity.

MOTIVATION

To demonstrate that human-GenAI interaction constitutes a new transmission paradigm, we conduct a measurement study on four platforms from major LLM providers⁸: *ChatGPT* (OpenAI), *Copilot*⁹ (Microsoft), *Gemini*¹⁰ (Google), and *Meta AI* (Meta). We focus on the voice calling scenario where the user sends an audio query to the LLM backend, which returns an audio response.

We analyze the following metrics: (1) the protocols used in transport/application layers; (2) the inter-arrival time between consecutive packets. For chunk-based delivery, this is defined as the interval between successive chunks; (3) packet size; and (4) throughput.

Protocols. We observe three distinct protocol stacks across the evaluated applications. *ChatGPT* and *Meta AI* adopt UDP/RTP, while *Copilot* employs TCP/H2. *Gemini* performs A/B testing by alternating between two schemes on different days, without requiring client-side updates. One variant uses TCP/H2 and the other uses QUIC with HTTP/3 (H3). We denote them as *Gemini-T* and *Gemini-Q*, respectively.

Uplink. Figure 2(a) presents the inter-arrival times of uplink packets across the evaluated applications. We observe that *ChatGPT*, *Gemini-Q*, and *Meta AI* exhibit short inter-arrival times (20–60 ms). These values correspond to the typical frame durations of the widely used audio codec,¹¹ suggesting these platforms employ frame-based transmission. We also observe that the downlink of *Gemini-Q* issues an acknowledgment (ACK) for each uplink packet, an approach generally avoided in interactive streaming systems due to the added latency and bandwidth overhead it imposes. Such behavior may be attributed to the intrinsic design of the QUIC protocol, which we discuss further in the section “*Direction-Dependent Delivery Method*.”

On the other hand, *Copilot* and *Gemini-T* exhibit larger inter-arrival times, indicative of a chunk-based delivery approach. For *Copilot*, the inter-arrival time consistently falls within the range of 150–160 ms, suggesting a fixed batching strategy that aggregates a set number of audio frames before transmission. In contrast, *Gemini-T* shows substantial variability in inter-arrival times, ranging from 100 to 400 ms.

The distinction between frame-based and chunk-based transmission methods can also be observed in the packet size distribution, as shown in Figure 2(b). Applications adopting frame-based methods generate small packets, with over 80% of packets less than 500 bytes. In contrast, applications using chunk-based approaches have larger packet sizes, with more than 60% of packets over 1,000 bytes, given that they aggregate multiple frames before transmission.

Downlink. Figure 2(c) shows the packet size distribution in the downlink, which helps us identify the underlying delivery method, as discussed above. We find that only *Gemini-Q* employs a different transmission strategy, chunk-based transmission for the downlink and frame-based delivery for the uplink. In contrast, all other applications

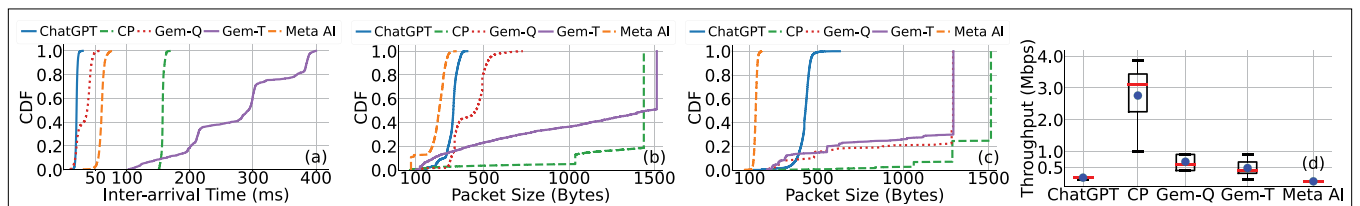


FIGURE 2. a) and b) Packet inter-arrival time and packet size of the uplink when the user is speaking. c) and d) Packet size and throughput of the downlink when the GenAI responds during human-GenAI calling in *ChatGPT*, *Copilot*, *Gemini* over QUIC, *Gemini* over TCP, and *Meta AI*.

⁸ <https://zapier.com/blog/best-llm/> (accessed on 30-September-2025)

⁹ <https://copilot.microsoft.com/> (accessed on 30-September-2025)

¹⁰ <https://gemini.google/overview/gemini-live/> (accessed on 30-September-2025)

¹¹ <https://datatracker.ietf.org/doc/html/rfc6716> (accessed on 30-September-2025)

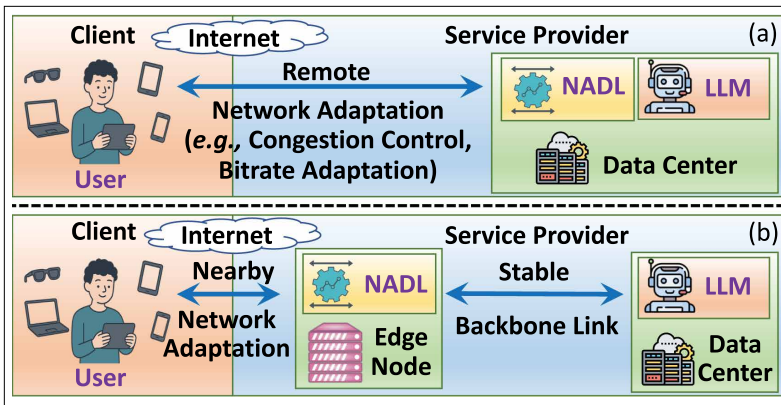


FIGURE 3. Comparison between a) conventional design and b) leveraging edge for network adaptation. For clarity, in a), we omit the edge node used as a simple forwarding relay, as it does not affect the fact that NADL operates over the extended path between the client and the data center.

maintain consistent delivery paradigms between uplink and downlink.

Different transmission schemes have a profound impact on downlink throughput, as illustrated in Figure 2(d). Chunk-based approaches generally consume substantially higher bandwidth than frame-based ones. For example, *Copilot* consumes ~3 Mbps on average, whereas *ChatGPT* and *Meta AI* maintain ~0.05 Mbps throughput, a 60× difference. As discussed in the section “Background,” this discrepancy arises because a large portion of LLM-generated responses is often available before transmission begins. This enables chunk-based approaches to aggressively exploit available bandwidth, downloading the response in bulk rather than delivering frame by frame.

Takeaways. We observe three distinct transmission paradigms across the four platforms studied: 1) both uplink and downlink adopt a frame-based approach over UDP/RTP (*ChatGPT* and *Meta AI*); 2) both directions rely on chunk-based delivery over TCP/H2 (*Copilot* and *Gemini-T*); and 3) the uplink employs frame-based delivery while the downlink uses chunk-based transmission over QUIC/H3 (*Gemini-Q*). These architectural choices have a profound impact on system performance, such as latency (which can be reflected in inter-arrival time) and bandwidth consumption. The diversity in protocol design highlights the absence of a clear consensus on the most suitable transmission paradigm for human-GenAI interaction.

RESEARCH AGENDA

In this section, we present our proposed architecture for optimizing human-GenAI interaction, as shown in Figure 1(b).

LEVERAGING EDGE FOR NETWORK ADAPTATION

Traditionally, network adaptation such as congestion control [1] has been *deprioritized* in audio streaming (e.g., human-to-human calling) due to the low bandwidth demands. However, this assumption no longer holds in the context of human-GenAI interaction. As shown in Figure 2(d), GenAI responses can drive downlink audio traffic to an average of ~3 Mbps, which necessitates explicit network adaptation [1], [9].

In multimedia systems, such adaptation is handled by the network adaptation layer (NADL), a dedicated component responsible for managing content delivery under varying network conditions through mechanisms such as congestion control [1].

To incorporate NADL in human and GenAI interaction, a straightforward architecture would position it within the data center where LLM inference takes place [12]. This design is generally considered practical, as it avoids introducing additional nodes or transmission hops that could increase end-to-end latency and infrastructure overhead. However, in the context of human-GenAI interaction, this architecture faces a critical limitation: the number of data centers capable of supporting real-time LLM inference is fundamentally constrained due to two key factors. First, LLMs are inherently resource-intensive, requiring substantial computational power. Moreover, to support real-time interaction, they require high-performance GPUs to rapidly generate responses. Consequently, LLMs for this use case are typically deployed only in regions with abundant and cost-effective compute resources. Second, many commercial-grade GPUs are subject to export control, making deployment infeasible in certain countries.¹² These two challenges, resource centralization and regulatory constraints, unfortunately, are non-trivial and unlikely to be resolved in the near future.

This design leads to an undesirable outcome: since only a limited number of data centers are capable of hosting LLMs, many users will inevitably be geographically distant from both the LLM and the co-located NADL, as shown in Figure 3(a). Moreover, communication between the client and the data center traverses the public Internet, which is often unstable (e.g., packet loss [13]). The combination of long geographic distance and poor Internet conditions results in high RTT [6] between the client and the NADL. To make matters worse, this high RTT severely undermines the effectiveness of NADL. Given that the Internet link is unstable, NADL must frequently react to changing conditions. However, adaptation algorithms are highly sensitive to RTT, as longer delays impair the timely delivery of feedback, making it difficult for NADL to swiftly respond to network fluctuations. Therefore, co-locating NADL with the LLM backend in data centers is suboptimal for human-GenAI interaction. One might consider applying a CDN-style caching mechanism to mitigate this issue. Unfortunately, as we discuss in the section “Background,” LLM-generated responses are non-cacheable.

To address the aforementioned limitations, we propose positioning the NADL to the network edge, as illustrated in Figure 3(b). In our architecture, the edge node serves as an intelligent relay between the user and the LLM back-end, hosting the NADL to manage transmission dynamics. Importantly, this can be achieved by reusing existing edge infrastructure, such as nodes already deployed for CDN services, thereby avoiding additional deployment overhead. A key insight of such a design is that while the end-to-end path includes both the client ↔ edge and edge ↔ data center segments, only the first segment is the bottleneck link, given that it traverses the

¹² <https://tinyurl.com/azdc86nd> (accessed on 30-September-2025)

public Internet. The second segment, by contrast, traverses high-throughput, stable backbone networks managed by service providers [7]. By positioning the NADL to the edge, network adaptation can focus on the short client $\leftrightarrow$ edge segment [14], where it is most needed, rather than being stretched across the entire path. As we demonstrate in the section “Impact of RTT on Congestion Control Performance,” this design significantly enhances the performance of network adaptation (e.g., congestion control) under lossy networks.

DIRECTION-DEPENDENT DELIVERY METHOD

As previously discussed in the section “Background” and empirically demonstrated in the section “Motivation,” human-GenAI interaction exhibits distinct asymmetric transmission characteristics between the uplink and downlink. In this subsection, we explore suitable transmission paradigms for each direction and discuss protocol designs that effectively support these asymmetric paradigms.

Downlink. A fundamental distinction of human-GenAI interaction from traditional human-human communication (e.g., voice/video calls) is the nature of the downlink, which carries LLM responses. They are token-based and often produced in bursts [8], allowing for two possible transmission paradigms: a frame-based approach akin to RTC or a chunk-based approach similar to on-demand content delivery. Both paradigms are currently employed in industry practices (the section “Motivation”). We summarize the comparative strengths and weaknesses of these two approaches in Table 1, detailed further below.

Bandwidth Cost/Utilization. As illustrated in Figure 2(d), frame-based applications (*ChatGPT* and *Meta AI*) transmit content frame by frame, resulting in low bandwidth consumption. However, this approach might underutilize available network resources. Conversely, chunk-based applications (*Copilot* and *Gemini*) aggressively utilize available bandwidth to quickly download generated content, thereby maximizing throughput. The optimal choice between frame-based and chunk-based methods thus depends on network conditions: frame-based delivery is preferable in bandwidth-constrained scenarios, while the chunk-based transmission excels under conditions of ample bandwidth availability.

Resilience to Packet Loss. Packet loss remains a pervasive challenge in Internet media delivery [13]. Frame-based transmissions are especially vulnerable because the loss of a frame can disrupt playback continuity [13]. In contrast, chunk-based transmissions are inherently more resilient. By utilizing higher bandwidth, they quickly complete content downloads, minimizing exposure to potential losses. Moreover, even if packet loss occurs during transmission, the previously downloaded data effectively serves as a buffer. As long as the download rate exceeds the playback rate, user-perceived playback continuity remains uninterrupted. Concurrently, the transport layer can keep retransmitting lost packets to continue delivering the remaining content.

Adaptation to New Session. Human-GenAI interactions may frequently involve abrupt user-driven topic changes, a phenomenon less

Property	Frame	Chunk
Bandwidth Cost/Utilization	Low	High
Resilience to Packet Loss	Low	Medium-High
Adaptation to New Session	Fast	Slow-Moderate

TABLE 1. Comparison of frame-based chunk-based approach for transmitting GenAI responses.

prevalent in traditional human-human communications. For instance, users may interrupt ongoing responses when they perceive that the GenAI’s reply deviates from their intent or when encountering common “LLM hallucinations.”¹³ Thus, quick adaptation to such interruptions is essential. Frame-based methods, which transmit content incrementally in small data units frame by frame, can quickly terminate ongoing transmissions and seamlessly transition to new content. Thus, chunk-based methods require transmitting large volumes of data per batch, which must be buffered across multiple layers. If the user interrupts during an ongoing transfer, the partially transmitted data becomes unusable, wasting bandwidth and resources. Moreover, coordinating cancellation across these layers takes time, potentially delaying the system’s responsiveness to new inputs.

Adaptive Delivery Selection. The analysis above highlights that the optimal content delivery strategy significantly depends on both network conditions and the specific context of human-GenAI interactions. For high-throughput, stable data center-to-edge connections, the chunk-based method is preferable because it can effectively utilize available bandwidth, ensuring rapid content transfer to the edge. Conversely, for edge-to-client connections, it is essential to account for dynamic network conditions and user interaction patterns.

To enable intelligent switching between chunk-based and frame-based transmission on the edge-to-client link, we propose an adaptive delivery controller deployed at the NADL. The goal of this controller is to estimate, at any moment during an ongoing session, which delivery mode will provide the best user experience and to adjust the transmission strategy accordingly. Designing such a controller requires addressing two fundamental issues: the selection of input signals and the choice of prediction model.

The first challenge is determining which signals to use as model inputs. Our earlier analysis indicates that the optimal delivery strategy depends not only on canonical network conditions but also on behavioral factors that arise specifically in human-GenAI interaction. Network-related inputs may include available bandwidth, packet-loss rates, RTT fluctuations, and retransmission statistics (observed during chunk-based transfers). Interaction-related inputs, on the other hand, capture user- and task-specific behavior such as interruption frequency and response length. However, since the NADL runs on resource-constrained edge infrastructure and must make decisions under tight latency budgets, it is difficult to incorporate every potentially useful metric.

The second challenge concerns the prediction model itself. Because these input signals naturally form time-evolving sequences, models capable

¹³ [https://en.wikipedia.org/wiki/Hallucination_\(artificial_intelligence\)](https://en.wikipedia.org/wiki/Hallucination_(artificial_intelligence)) (accessed on 30-September-2025)

of capturing temporal structure, such as recurrent neural networks (RNNs) or Transformers, are promising candidates. Yet these models must remain sufficiently lightweight to meet real-time inference requirements at the edge. Large or deep architectures with high parameter counts are therefore impractical. The above two issues require a careful co-design: selecting input features that minimize redundancy while preserving predictive value, and choosing a model architecture that balances temporal expressiveness with computational efficiency to ensure timely delivery-mode decisions.

To achieve this, we will construct a large-scale dataset through user studies in controlled environments. For each experiment, we sample an input query from the established benchmark [2]. The participant then issues this query twice. The GenAI response on the edge-to-client link is delivered via chunk-based transmission for one trial and via frame-based transmission for the other. After experiencing both variants, the participant provides mean opinion scores (MOS) for each, which we treat as the ground-truth QoE labels for the two delivery strategies. Simultaneously, the system logs all relevant network and interaction signals. These paired observations allow us to evaluate different combinations of input features and model architectures, identifying configurations that achieve high QoE-prediction accuracy while remaining lightweight enough for real-time edge deployment. Once trained, the model is deployed at the NADL and evaluated periodically during live sessions. The NADL uses recently observed network and interaction signals as input to the model, queries the predicted QoE for both chunk-based and frame-based transmission, and selects the mode expected to yield superior performance.

Uplink. The uplink carries live human speech/video that must be streamed in real time, making it well-suited for the frame-based method employed in RTC because it can quickly transmit the available data to servers. For instance, as shown in Figure 2(a), at the 80th percentile, *Gemini-T* using chunk-based methods exhibits more than 300 ms higher inter-arrival times compared to *Gemini-Q*, which employs frame-based solutions. This increased latency contributes significantly to the overall end-to-end delay. Thus, the entire uplink should adopt frame-based approaches.

Note that this latency discrepancy may not exist in the downlink. This is because user-perceived latency for GenAI responses depends on the playback timing of the first token, marking the start of the AI's response. Thus, both frame-based and chunk-based solutions can be designed to transmit the first token and start playback as soon as it is available.

Protocols. To effectively support the heterogeneous transmission strategy discussed above, transport/application protocols must explicitly accommodate different delivery schemes and facilitate seamless switching between them. This requirement challenges conventional symmetric transmission protocols such as UDP/RTP and TCP/H2, as indicated in the section “Motivation.”

QUIC/H3 represents a promising underlying protocol due to its versatility and extensibility, as deployed in *Gemini-Q* (the section “Motivation”).

However, we observe that *Gemini-Q* sends an acknowledgment (ACK) for every frame on the uplink path.

This behavior largely stems from QUIC's reliability-oriented design, which mandates ordered delivery and ACK-based loss detection.¹⁴ Such mechanisms, while essential for reliable data transfer, introduce retransmissions and enforce in-order delivery. These behaviors are generally undesirable in RTC systems, where timely delivery outweighs strict reliability. To mitigate this, one promising approach is to adopt the emerging Media-over-QUIC (MoQ) paradigm¹⁵ that delivers the media using QUIC's unreliable datagram extension.¹⁶ As we demonstrate in the section “Comparison of MoQ and Native QUIC,” MoQ significantly reduces latency under lossy conditions compared to native QUIC. Simultaneously, the native QUIC protocol can be retained for chunk-based downlink transmissions. This approach facilitates a unified transport layer that dynamically adapts its delivery mode without introducing excessive protocol overhead and latency.

PRELIMINARY RESULTS

In this section, we first present a proof-of-concept implementation to demonstrate the effectiveness of placing the NADL at the network edge for improved network adaptation (the section “Leveraging Edge for Network Adaptation”). Our primary goal is to validate that, under lossy conditions, congestion control performs significantly better on low-RTT paths than on high-RTT ones. We then compare the performance of MoQ and native QUIC for uplink real-time media delivery to demonstrate the suitability of using MoQ for uplink transmission (the section “Direction-Dependent Delivery Method”). Our experiments show that MoQ's unreliable transmission maintains stable latency even under significant packet loss, whereas native QUIC's reliable transmission leads to sharp latency degradation.

IMPACT OF RTT ON CONGESTION CONTROL PERFORMANCE

Experiment Setup. For controlled experiments, we emulate the transmission using two virtual network interfaces on a single Linux machine: one representing the client, and the other for the NADL. We use Linux `tc`¹⁷ to configure the RTT between interfaces, varying it from 10 ms, reflecting the best-case scenario where the NADL is located at a nearby edge node [14], to 1,000 ms, which approximates a worst-case scenario where the user is geographically distant from the data center [6], combined with internal backend delays [7]. To simulate realistic loss conditions prevalent on the Internet [13], we use cellular network loss traces released by Akamai collected from its CDN infrastructure¹⁸ The loss patterns of these traces are *bursty*, for instance, instantaneous rates exceeding 80%, a characteristic commonly observed in practice [13].

We evaluate three popular congestion control algorithms: CUBIC and Reno, both deployed in QUIC [11], as well as GCC, widely used in RTC systems [10]. We use `aioquic`¹⁹ to implement QUIC-based transmission with CUBIC and Reno, and `aiortc`²⁰ to implement RTC-based transmission with GCC. The target sending bitrate is set to 3 Mbps, which aligns with the average bitrate

¹⁴ <https://datatracker.ietf.org/doc/html/rfc9002> (accessed on 30-September-2025)

¹⁵ <https://datatracker.ietf.org/group/moq/about> (accessed on 30-September-2025)

¹⁶ <https://datatracker.ietf.org/doc/html/rfc9221> (accessed on 30-September-2025)

¹⁷ <https://linux.die.net/man/8/tc> (accessed on 30-September-2025)

¹⁸ <https://github.com/akamai/cell-emulation-util> (accessed on 30-September-2025)

¹⁹ <https://github.com/aiortc/aioquic> (accessed on 30-September-2025)

²⁰ <https://github.com/aiortc/aiortc> (accessed on 30-September-2025)

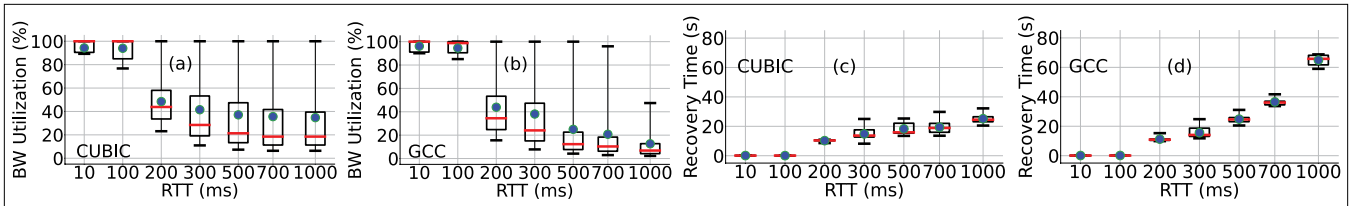


FIGURE 4. Bandwidth utilization during loss for CUBIC a) and GCC b), and recovery time after loss for CUBIC c) and GCC d), under the packet loss traces provided by Akamai.

GenAI responses could consume, as shown in Figure 2(d). This fixed target bitrate enables fair comparison across different setups, a method commonly used in previous efforts [11].

For each experiment, we proceed in three phases: 1) wait until the sending bitrate reaches the target; 2) apply a 90-second loss trace; 3) terminate the induced loss. Each configuration is repeated 20 times. We report two key metrics: *Bandwidth Utilization During Loss*. When packet loss is detected, congestion control algorithms reduce the sending bitrate to avoid overwhelming the network. We measure bandwidth utilization as the ratio of the sending bitrate to the target bitrate during loss periods. This metric itself is important for CDN-style delivery, and it also indicates throughput stability over time, which is crucial for RTC-style transmission (the section “Background”).

Recovery Time. We measure how long it takes for the sender to achieve the target bitrate after the loss condition is terminated. This reflects the responsiveness of the congestion control in adapting from lossy to loss-free conditions.

Experimental Results. Figure 4 presents the performance of CUBIC and GCC across varying RTTs. We omit the results of Reno as it is similar to GCC. The box plot shows the 95th, 75th, 25th, and 5th percentiles, median, and mean (blue dots). In the following analysis, we use (a)–(d) to denote the subfigures in Figure 4.

We observe that as RTT increases, the performance of congestion control degrades: bandwidth utilization drops, bandwidth variability increases, and recovery time lengthens. This degradation is due to the inherent sensitivity of congestion control to RTT [11]. Notably, under 100-ms RTT, the performance of CUBIC and GCC remains good and comparable to the 10-ms RTT. Specifically, their average and median bandwidth utilization exceed 90%, with small variation (e.g., 5th percentile remains above 80%), presented in (a) and (b). Furthermore, both algorithms achieve instantaneous recovery after loss, shown in (c) and (d). In contrast, once RTT increases beyond 200 ms, performance degrades significantly.

Both CUBIC and GCC drop to around 40% average and median bandwidth utilization, and recovery time extends to ~10s. This degradation becomes more severe at higher RTTs. For instance, at 1000-ms RTT, GCC achieves only <20% bandwidth utilization on both the average and 75th percentile (b), presented in (b), and takes over 60 seconds to recover on average, as shown in (d).

Takeaways. Our results demonstrate that, regardless of the congestion control algorithm or transmission paradigm, lower RTT consistently

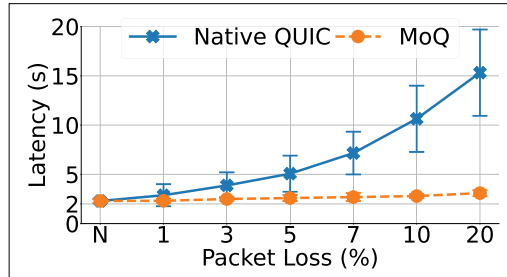


FIGURE 5. Conversational latency using native QUIC and Media over QUIC (MoQ) under different packet loss rates. N: No packet loss.

yields better performance under lossy conditions. When RTT is below 100 ms, which represents the typical upper bound for the latency between user and the edge [14], the system maintains high bandwidth utilization, low variability, and instantaneous recovery from loss. In contrast, once RTT exceeds 200 ms, commonly observed between client and remote data centers [6], performance degrades significantly across all metrics. These results underscore the necessity of positioning the NADL at the network edge to enable fast, robust network adaptation.

COMPARISON OF MoQ AND NATIVE QUIC

Experiment Setup. To evaluate performance under different transmission methods (MoQ and native QUIC), we build an end-to-end human-GenAI voice-calling testbed. The system takes live user audio as input and returns a GenAI-generated audio response. The backend pipeline follows the popular processing architecture of existing commercial platforms [2], composed of automatic speech recognition (ASR), LLM inference, and text-to-speech (TTS) synthesis. The backend logic is implemented using the OpenAI API.²¹

We implement both MoQ-based transmission (using QUIC datagrams) and native QUIC transmission using `aioquic`. To examine their performance in degraded network conditions, following the prior measurement study on RTC [5], we configure both uplink and downlink packet loss rates to {1, 3, 5, 7, 10, 20}%. We report *conversational latency*, defined as the user-perceived end-to-end delay between the moment the user finishes speaking and the moment the user first hears audio responses from GenAI.

Experimental Results. As shown in Figure 5. MoQ’s conversational latency remains almost unchanged even under high packet loss. For instance, at 20% loss, its latency increases by less than 1s relative to no-loss conditions. In contrast, native QUIC experiences substantial degradation:

²¹ <https://openai.com/api/> (accessed on 30-September-2025)

its latency begins to rise noticeably beyond 3% loss (e.g., ~5s at 5% loss) and becomes severely inflated at higher loss rates (e.g., over 15s at 20% loss). This discrepancy arises from QUIC's reliable transmission manner, which mandates in-order delivery and triggers retransmissions upon loss. These behaviors induce head-of-line (HOL) blocking,²² delaying subsequent packets until missing data is retransmitted and received.

Takeaways. These results show that unreliable media delivery via MoQ substantially reduces conversational latency compared to native QUIC under packet loss when streaming live uplink media in human-GenAI interaction. This finding reinforces our proposal to use QUIC as the underlying transport protocol for asymmetric transmission, specifically employing MoQ for frame-based uplink live media and native QUIC for chunk-based downlink GenAI responses. As a result, the system retains a unified transport protocol while tailoring the transmission method for each path to its distinct characteristics and requirements.

DISCUSSION

Implementation Complexity of NADL. Implementing NADL in practice is non-trivial. For instance, even though NADL can reuse existing CDN infrastructure (the section “Leveraging Edge for Network Adaptation”), assigning appropriate NADL nodes to individual users remains challenging. Traditional CDN server selection is optimized for the instantaneous available bandwidth to maximize utilization for chunk-based content delivery (the section “Background”), which aligns with the downlink of human-GenAI interaction. However, uplink traffic is RTC-style and carried via frame-based transmission, where bandwidth stability is critical. As such, directly reusing CDN selection logic may choose nodes that maximize throughput but exhibit undesirable short-term instability for interactive uplink traffic. This calls for new NADL selection strategies that jointly account for both uplink stability and downlink capacity. Moreover, the system may need to migrate an ongoing session to a different NADL when the network conditions of the current client ↔ NADL path degrade, raising challenges such as when to trigger migration, how to select suitable alternative NADL nodes, and how to switch without disrupting the underlying inference session. All of these issues warrant further investigation.

Managing Resource Contention Within a Single Connection. A single human-GenAI session may also experience contention for server-side processing resources on the server side. Modern GenAI calling pipelines typically interleave automatic speech recognition (ASR), LLM inference, and text-to-speech (TTS) generation, all of which share CPU, GPU, and memory resources and operate in a pipelined manner over the same session context [2]. When the downlink delivers large bursts of generated audio, the system must decode, buffer, and schedule these outputs while simultaneously ingesting and processing uplink audio/video frames. This can create competition for compute cycles, memory bandwidth, and buffer capacity, causing queuing delays that inflate conversational latency. Likewise, periods of intensive uplink processing for live audio/video capture

can interfere with the timely synthesis of downlink responses. Addressing this challenge calls for resource scheduling approaches that are aware of the entire pipeline within a single session, such as prioritizing latency-critical stages (e.g., the early part of TTS) to maintain a low-latency interactive experience.

Deployment Trade-Offs of NADL. Deploying NADL also raises several trade-offs that must be judiciously balanced. One challenge lies in deciding how densely NADL should be deployed. A denser footprint increases the likelihood that a user connects to a nearby NADL, reducing RTT variation and improving stability on the client ↔ NADL bottleneck, but increases operational cost and may be infeasible in regions with limited edge presence. In contrast, sparse deployments lower cost but create geographic disparities with users far from NADL experiencing consistently higher RTT and slower recovery under network disruption. Another deployment trade-off arises from how operators assign users to NADL under varying load conditions. Aggressively multiplexing many concurrent sessions onto the same NADL improves infrastructure efficiency but may cause queue buildup or overload, whereas conservative assignment improves stability at the expense of lower utilization. These trade-offs motivate research on NADL assignment policies that incorporate remaining capacity, predicted load, and path stability to provide consistent interactive quality across diverse network environments.

Supporting Coexistence of Human-to-Agent and Agent-to-Agent Interactions. Looking forward, we expect GenAI services to support both real-time human-to-agent communication and agent-to-agent orchestration,²³ where multiple GenAI agents collaborate on a complex task. These two classes of interactions have fundamentally different requirements. As discussed throughout the paper, human-to-agent flows combine RTC-like uplink and CDN-like downlink delivery. In contrast, agent-to-agent exchanges are typically higher-volume, bursty, and more tolerant to latency, resembling CDN-style chunk transfers between servers (e.g., across data centers). As a result, the end-to-end GenAI infrastructure, including NADL nodes at the edge and back-end data centers, must jointly accommodate RTC-like uplink traffic and CDN-like downlink traffic for human-facing content, as well as CDN-style inter-agent coordination traffic. Managing these heterogeneous flow classes within a shared infrastructure raises practical challenges, such as deciding which traffic should be prioritized, where to enforce isolation between flow classes, and how to prevent interference between human-facing and orchestration workloads. These challenges merit further investigation.

CONCLUSION

In this position paper, we proposed a holistic network architecture to support real-time human and GenAI interaction. Our design is motivated by a measurement study and careful analysis of current deployments, which reveal that this emerging paradigm demands low latency, resilience to diverse network conditions, and support for asymmetric transmission behavior,

²² https://en.wikipedia.org/wiki/Head-of-line_blocking (accessed on 30-September-2025)

²³ <https://www.talkdesk.com/blog/multi-agent-orchestration/> (accessed on 30-September-2025)

requirements that existing efforts struggle to meet. To address these challenges, we first introduce an intelligent relay at the edge to enable fast and robust network adaptation, followed by designing a unified transport protocol with a direction-specific delivery framework that adapts transmission strategies based on each link's characteristics. We hope this work inspires future research on optimizing networked systems for GenAI-driven communication.

ACKNOWLEDGMENT

No Meta resources were used for this article. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Meta.

REFERENCES

- [1] G. Carlucci et al., "Congestion control for web real-time communication," *IEEE/ACM Trans. Netw.*, vol. 25, no. 5, pp. 2629–2642, Oct. 2017.
- [2] R. Cheng et al., "Hello, GenAI? Dissecting human to generative AI calling," in *Proc. ACM Internet Meas. Conf.*, Oct. 2025, pp. 308–324.
- [3] R. Cheng et al., "MagicStream: Bandwidth-conserving immersive telepresence via semantic communication," in *Proc. 22nd ACM Conf. Embedded Networked Sensor Syst.*, Nov. 2024, pp. 365–379.
- [4] R. Cheng et al., "A first look at immersive telepresence on apple vision pro," in *Proc. ACM Internet Meas. Conf.*, Nov. 2024, pp. 555–562.
- [5] R. Cheng et al., "Are we ready for metaverse? A measurement study of social virtual reality platforms," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 504–518.
- [6] T. K. Dang et al., "Cloudy with a chance of short RTTs: Analyzing cloud connectivity in the Internet," in *Proc. 21st ACM Internet Meas. Conf.*, Nov. 2021, pp. 62–79.
- [7] J. Li et al., "LiveNet: A low-latency video transport network for large-scale live streaming," in *Proc. ACM SIGCOMM Conf.*, Aug. 2022, pp. 812–825.
- [8] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 38–56.

- [9] K. MacMillan et al., "Measuring the performance and network utilization of popular video conferencing applications," in *Proc. 21st ACM Internet Meas. Conf.*, Nov. 2021, pp. 229–244.
- [10] Z. Meng et al., "Enabling high quality real-time communications with adaptive frame-rate," in *Proc. USENIX NSDI*, 2023, pp. 1429–1450.
- [11] A. Mishra, S. Lim, and B. Leong, "Understanding speciation in QUIC congestion control," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 560–566.
- [12] K. Qian et al., "Alibaba HPN: A data center network for large language model training," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 691–706.
- [13] M. Rudow et al., "Tambur: Efficient loss recovery for videoconferencing via streaming codes," in *Proc. NSDI*, 2023, pp. 953–971.
- [14] M. Xu et al., "From cloud to edge: A first look at public edge platforms," in *Proc. 21st ACM Internet Meas. Conf.*, Nov. 2021, pp. 37–53.
- [15] H. Yu et al., "Magpie: Improving the efficiency of A/B tests for large scale video-on-demand systems," in *Proc. ACM IMC*, 2024, pp. 588–594.

BIOGRAPHIES

RUIZHI CHENG (ruizhi.cheng@meta.com) is a research collaborator at Meta Platforms, Inc. He is currently pursuing the Ph.D. degree at George Mason University. His current research focuses on building large-scale, learning-based, real-time intelligent networked systems for emerging communication paradigms such as immersive telepresence and human-GenAI interaction. He received the Best Paper Runner-up Award from ACM SenSys 2024.

GUOWU XIE (woo@meta.com) received the Ph.D. degree in computer science from the University of California, Riverside. He is a Research Scientist at Meta Platforms Inc. His work focuses on designing, implementing, and operating large-scale networking software systems, ensuring seamless connectivity and optimal performance for billions of users worldwide. He remains actively engaged with academic and professional communities, regularly sharing insights and best practices in large-scale distributed systems.

BO HAN (bohan@gmu.edu) is currently an Associate Professor at the Department of Computer Science, George Mason University, previously a Principal Research Scientist at AT&T Labs. He has authored more than 100 papers, holds more than 60 U.S. patents, and received multiple best paper awards. His research focuses on networked systems and mobile computing.